1 The story in one minute

1. A **solar plant** has an *inverter*. It turns sunlight into electricity, and you can tell it how hard to work: 100 % means full power, 40 % means 40 kW of a 100 kW plant, 0 % means stop.
2. A company far away, the **direct marketer**, sells that electricity. It wants to *read* the plant's values and *send limits*.
3. The marketer speaks one language (**IEC 104**), the inverter another (**Modbus**). They cannot understand each other.
4. Our project is the **translator in the middle**: the **gateway**.

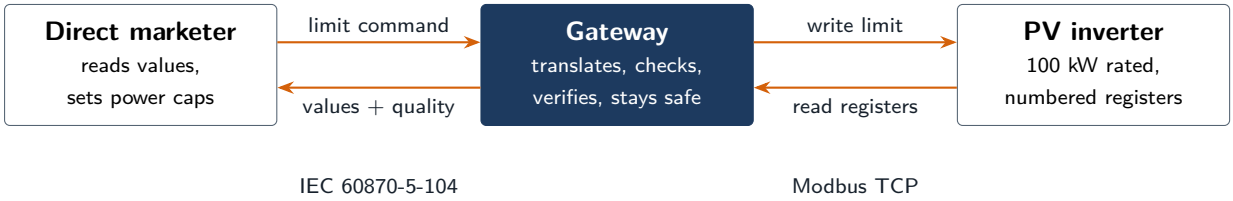


Figure 1. The gateway is the only part that speaks both languages. Both the inverter and the marketer are simulated in this project.

The one rule behind everything: *a wrong value is worse than no value.* When the gateway is not sure, it says "invalid" or "rejected" instead of guessing.

1.1 The five jobs (requirements)

	Job in plain words	Section
R1	Ask the inverter every second and translate the numbers into the marketer's units.	2
R2	Do not pass on old news as if it were fresh.	3
R3	Handle the marketer's orders carefully: check, write, verify.	4
R4	Decide what is safe when the phone line to the marketer dies.	5
R5	Prove with automatic tests that all of this works.	README

2 R1: ask every second and translate

The inverter keeps its numbers in numbered boxes called *registers*. Every second the gateway reads boxes 40000 to 40005 in one request. Each box holds only a small number (16 bits), so a big number such as power uses **two boxes** that must be joined.

Why two boxes? 40000 holds the upper half of the power number and 40001 the lower half. The task lists only 40000 because the second half sits right behind it, and likewise 40002 and 40003.

Why the sign matters. Reactive power can be negative. If the two halves were joined without a sign, −1200 would turn into +4 294 966 096. A test checks that −1 comes out as −1.

Strange values are refused, not guessed. A status of 42 does not exist (only 0 to 3), so it is

Table 1. One real poll: what the inverter says, and what the marketer gets.

Boxes	Meaning	Raw number	Sent to marketer
40000 + 40001	Active power (joined)	50 000 W	50.0 kW
40002 + 40003	Reactive power (joined)	−1200 var	−1.2 kvar
40004	Grid frequency	5001 (steps of 0.01 Hz)	50.01 Hz
40005	Status	1	1 = running
40010	Active power limit	100	100 %

not stored.

3 R2: do not pass on old news

Every value is stored together with the time it arrived. When the gateway publishes a value it first works out the age:

$$\text{age} = \text{now} - \text{time the value arrived} \qquad \text{age} \le 5\,\text{s}: \textbf{ valid} \qquad \text{age} > 5\,\text{s}: \textbf{ invalid}$$

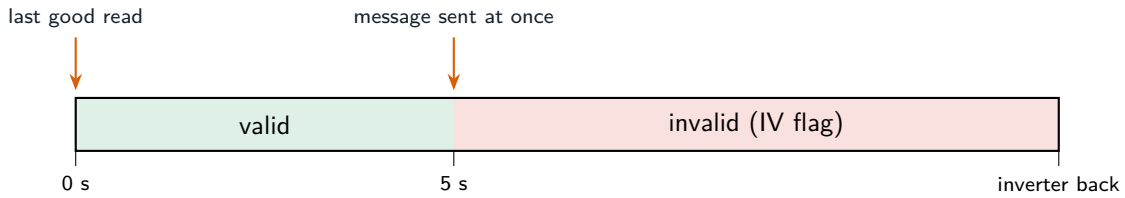


Figure 2. The inverter stops answering at 0 s. Up to 5 s the old value is still valid. After that it is flagged invalid until a new read succeeds.

Table 2. The inverter reads 50 kW, then stops answering at 0 s.

Time	Event	What the marketer sees
0 s	last good read	50 kW, good
3 s	poll fails	50 kW, good (age 3 s)
5.0 s	poll fails	50 kW, good (not older than 5 s)
5.1 s	poll fails	50 kW, **invalid**, sent at once
40 s	inverter answers again	new value, good

Why 5 seconds and not 1? The task says “older than 5 seconds”. One slow poll would otherwise flip the value back and forth. The limit is a setting (`stale_after_s`).

A failed read never refreshes the age. Only a successful read does.

What is sent for a point that is invalid? Always a number plus the flag. A point that was never received carries the placeholder 0.0 with the invalid flag, so a receiver that checks the flag never mistakes it for a measurement.

Known limit: Modbus has no timestamps. The gateway stamps the time a value *arrived*, not the time it was measured.

4 R3: handle orders carefully

Think of sending a letter and phoning to check that it arrived. Five steps:

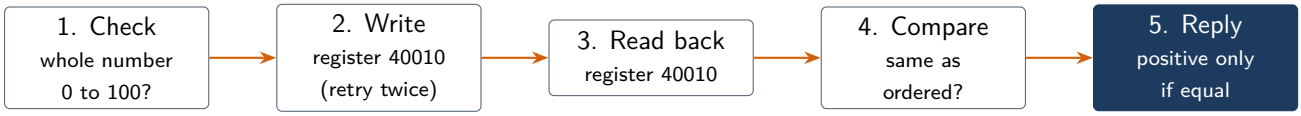


Figure 3. Any step that fails ends in a negative reply.

Table 3. What happens to different orders.

Marketer sends	What the gateway does	Reply
40	Valid. Writes 40, reads back 40: equal.	**positive**
150 or −5	Out of range. Nothing is written.	negative
50.5, NaN, text	Not a whole number. Nothing is written.	negative
40, but the inverter secretly applies 60	Writes 40, reads back 60: not equal. The limit shown is the real 60.	negative
40, but the cable is pulled	1 try plus 2 retries all fail. The limit point becomes invalid because the real limit is unknown.	negative

Why read back? A device can accept a write and still not apply it. Only the read-back shows what the plant really does.

5 R4: what if the phone line dies?

The gateway watches how long the marketer has been silent. A connected client, a command, and even a rejected command all count as a sign of life. After **more than 60 seconds** of silence the *fallback* runs. New activity cancels it.

Table 4. The marketer had set 40 %, then the line dies at 0 s.

Time	Policy `hold` (default)	Policy `safe_limit` with preset 30 %
60 s	nothing yet (not more than 60 s)	nothing yet
61 s	fallback flag set, limit stays **40 %**	reads current limit 40, writes **30 %**, verifies

Why `hold` is the default. The limit was a deliberate order. Going to 100 % could break a curtailment that is active exactly when the line fails. Going to 0 % loses money during what is usually a short outage. Holding never allows more than was last permitted. The weakness is that a stale low limit persists, so the fallback flag is exposed and a site can choose `safe_limit`.

5.1 Two safety rules for `safe_limit`

1. **The value must be chosen.** Without `-fallback-limit` between 0 and 100 the gateway refuses to start.
2. **It can only tighten:** new limit = the lower of (current limit, preset).

Table 5. Preset 30 % on a 100 kW plant.

Current limit	Result
100 %	100 is above 30: write 30. The limit becomes 30 % (30 kW).
20 %	20 is not above 30: write nothing. It stays 20 %; writing 30 would have added 10 kW.
cannot be read	Write nothing, because a blind write might raise the limit. Try again about 1 s later.

A failed write to the inverter is a separate case: the order gets a negative reply and the limit point becomes invalid. It does not start the fallback, because the line to the marketer is fine.

6 The two written questions, in short

6.1 What changes if the gateway controls the power itself?

Today the marketer decides and the gateway only passes the limit on. A *plant controller* decides: it keeps the power at the grid connection under a limit, continuously. That is harder because of (1) a closed loop with meter delay, (2) several devices sharing one limit, (3) certification under VDE-AR-N 4110, where a bug can break the grid code, (4) a fallback that needs a time limit and a defined value (holding alone is not enough), and (5) a priority order between marketer, grid operator and local limits.

6.2 How to test it on a real plant the first time?

(1) Read-only first, writes disabled in code. (2) Agree a window with an electrician on site and a rollback contact. (3) First write is a harmless value, then a small step, watching real output. (4) Break things on purpose: pull the cable, block the line, restart mid-limit. (5) Smallest plant first with a clamped range. (6) Shadow mode with the marketer reading only, before real orders.

7 What is missing for a real plant

1. IEC 104 is basic: no select-before-operate, no security, only the invalid flag.
2. Polling is simple: a slow inverter can exceed one second.
3. No check whether a value is physically plausible, only whether it is in range.
4. Nothing is saved across a restart.
5. One inverter, no login, no limit on command rate.
6. Never tested on real hardware (word order, scaling and write behaviour differ between devices).
7. Not a plant controller: no loop at the grid connection point.